TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC QUỐC GIA HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

NGHIÊN CỨU KIẾN TRÚC LAI HYBRID ATTENTION-SSM PHỤC VỤ CHUYỂN PHONG CÁCH VĂN BẢN VÀ DỊCH THUẬT HÁN - VIỆT

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Sinh viên báo cáo:
Hà Vũ Công (MSSV: 23020014)
Ngành Công nghệ Thông tin

Giảng viên hướng dẫn:
TS. Nguyễn Văn Vinh
Khoa Công nghệ Thông tin

Hà Nội, 2026

Nội dung Trình bày (Agenda)

A. Cơ sở Khoa học & Kiến trúc

- **Phần I: Đặt vấn đề & Động lực**
  - Đặc thù dịch Hán - Việt trường thiên.
  - Giới hạn Seq2Seq câu đơn.
  - Điểm nghẽn $O(N^2)$ & KV-Cache.
- **Phần II: Kiến trúc Lai Đề xuất**
  - Mô hình Evolving-Head Hybrid SSM.
  - Mô hình hóa 6 bước Tensor.
  - Linear Scan $O(N)$ & SSM $O(1)$.

B. Thực nghiệm & Ứng dụng

- **Phần III: Kỹ nghệ Huấn luyện**
  - Streaming 34.3M câu nhị phân.
  - Chunked Loss GPU 16GB.
- **Phần IV: Thực nghiệm Đối chứng**
  - Benchmark 200k steps với Transformer.
  - Mở rộng cực hạn 2,000,000 tokens.
  - Đánh giá BLEU & Định tính.
- **Phần V: Demo Thực tế & Kết luận**
  - Ứng dụng Web / CLI ngoại tuyến.
  - Đóng góp cốt lõi của đề tài.

Bối cảnh: Dịch thuật Hán - Việt & Văn bản Trường thiên

Đặc thù ngôn ngữ Hán - Việt:

- **Kho từ vựng cổ phong:** Chứa hàng vạn thuật ngữ tu tiên, kiếm hiệp, điển tích điển cố.
- **Cấu trúc ngữ pháp đa tầng:** Đòi hỏi chuyển dịch mượt mà nhưng phải giữ khí chất cổ kính.
- **Quy mô văn bản trường thiên:** Tiểu thuyết, chương truyện dài từ hàng chục nghìn tới hàng triệu tokens.

Nhu cầu cấp bách:

- Mô hình phải **ghi nhớ thực thể tên riêng** lâu dài.
- Vận hành bền bỉ trên **phần cứng biên / máy tính cá nhân**.

Thách thức Công nghệ Hiện nay

1. NMT Seq2Seq truyền thống:

- Bó hẹp ở cấp câu đơn (≤ 512 tokens).
- Đứt gãy ngữ cảnh, quên nhân vật khi dịch trường thiên.

2. Transformer Decoder-Only (LLMs):

- Chi phí chú ý bậc hai $O(N^2)$ quá lớn.
- Bùng nổ bộ nhớ KV-Cache $\Longrightarrow$ OOM trên phần cứng cá nhân!

Điểm nghẽn $O(N^2)$ & Bùng nổ Bộ nhớ KV-Cache

1. Độ phức tạp Bậc hai $O(N^2)$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Phép nhân ma trận $(N \times d) \times (d \times N) \rightarrow N \times N$.
- Chi phí tính toán: $O(N^2 \cdot d)$.
- Chi phí lưu ma trận Softmax: $O(N^2)$.

2. Nghịch lý Bùng nổ KV-Cache

$$\text{Mem}_{\text{KV}} = 2 \times B \times L \times H \times d_k \times \text{bytes} \times N$$

- Mỗi token mới sinh thêm: $+16$ KB/token.
- **Ở 100,000 tokens:** Chiếm 1.52 GB KV-Cache.
- **Ở 900,000 tokens:** Vượt ngưỡng bộ nhớ VRAM 16GB.
- **Ở 2,000,000 tokens: 30.52 GB VRAM $\Longrightarrow$ CUDA Out-Of-Memory!**

Cần một kiến trúc mới có khả năng duy trì trạng thái nhớ hằng số $O(1)$!

Kết hợp Ưu điểm Bổ trợ của 2 Hướng tiếp cận Kiến trúc

Nhánh 1: Micro-Head Linear Attention

- Ánh xạ Kernel dương: $\phi(x) = \text{ELU}(x) + 1.0$.
- Đảo thứ tự nhân ma trận qua tính chất kết hợp.
- Trạng thái tích lũy ma trận bậc hai:

$$S_t = S_{t-1} + \phi(k_t)^T v_t \in \mathbf{R}^{d_k \times d_k}$$

- **Ưu thế:** Truy hồi liên kết toàn cục với chi phí tuyến tính $O(N)$.

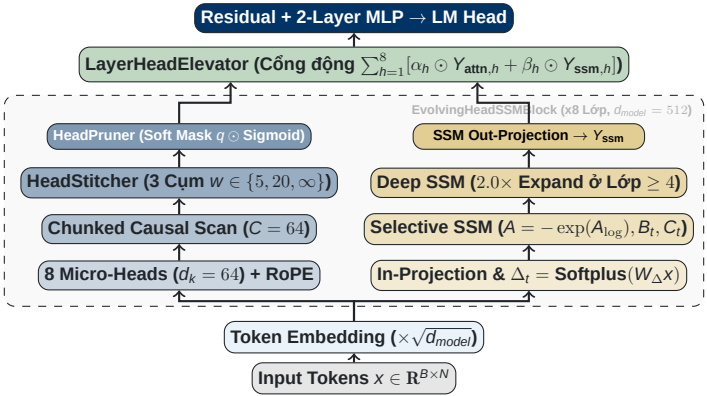
Nhánh 2: Selective State Space Model (SSM)

- Kế thừa nguyên lý chọn lọc Mamba.
- Tham số hóa phụ thuộc dữ liệu: $B_t, C_t, \Delta_t(x_t)$.
- Vector trạng thái ẩn liên tục:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t \in \mathbf{R}^{d_{inner} \times d_{state}}$$

- **Ưu thế:** Nén thông tin và lọc nhiễu với dung lượng nhớ cố định $O(1)$.

Sơ đồ Khối Tổng thể Kiến trúc Đề xuất (71.97M Tham số)



Mã nguồn mô hình tự cài đặt từ đầu (from scratch) bằng PyTorch: `models/micro_head_ssm.py`

Mô hình hóa 6 Bước Thực thi Tensor trong Model

Nhánh Linear Attention (Bước 1 $\rightarrow$ 3)

1. Input Scaling & RoPE 2D Rotation:

- $x_0 = \text{Embedding}(x) \cdot \sqrt{d_{model}}$ ($d_{model} = 512$).
- Chiếu Q, K, V (8 vi-đầu, $d_k = 64$).
- Xoay vị trí: $Q, K \leftarrow \text{RoPE}(Q, K, \cos, \sin)$.

2. Dynamic Stitching & Soft Pruning:

- 3 cụm $w \in \{5, 20, \infty\}$; $\mathcal{P} = \text{Softmax}(\text{Router}(x))$.
- Lọc vi-đầu nhiễu: $Q \leftarrow Q \odot \sigma(\text{Linear}_{imp}(x))$.

3. Chunked Linear Scan ($C = 64, O(N)$):

- Kernel dương: $\phi(x) = \text{ELU}(x) + 1.0 \geq 0$.
- Tích luỹ: $S^{(c)} = S^{(c-1)}\gamma_c + \phi(K_c)^T V_c$.

Nhánh Selective SSM & Kết hợp (Bước 4 $\rightarrow$ 6)

4. Selective SSM Recurrence (Mamba):

- $\Delta_t = \text{Softplus}(W_\Delta x_t)$, $A = -\exp(A_{\log}) < 0$.
- Train: Parallel Scan; Infer: Đệ quy $O(1)$.

5. Dynamic Layer Head Elevator:

- Cổng: $\beta_h = \sigma(\text{logit}_{base} + W_e x)$, $\alpha_h = 1 - \beta_h$.
- Pha trộn: $Y = \sum_{h=1}^{8} [\alpha_h \odot Y_{\text{attn},h} + \beta_h \odot Y_{\text{ssm},h}]$.
- Lớp ≥ 4: Mở rộng Deep SSM $2\times$ ($d_{state} = 1024$).

6. Residual, Gated MLP & Generator:

- $x \leftarrow x + Y + \text{MLP}(\text{LayerNorm}(x))$.
- Qua 8 lớp $\rightarrow$ FinalNorm $\rightarrow$ LM Head $\rightarrow$ Logits.

Cơ chế 1: Mã hóa Vị trí Quay RoPE (Rotary Position Embedding)

Hạn chế của Embedding vị trí tuyệt đối:

- Cộng thô trực tiếp vào vector nhúng từ: $x + p_i$.
- Không có khả năng ngoại suy khi chuỗi vượt kích thước huấn luyện.

Nguyên lý Xoay Không gian 2D của RoPE:

- Xoay vector Q và K theo góc tỷ lệ thuận với vị trí m:

$$R^d_{\Theta,m} = \text{diag}\left(R_{\theta_1,m}, R_{\theta_2,m}, \ldots, R_{\theta_{d/2},m}\right)$$

- Tích vô hướng chỉ phụ thuộc vào khoảng cách tương đối:

$$\langle R^d_{\Theta,m} q, R^d_{\Theta,n} k \rangle = g(q, k, m-n)$$

Lợi ích Kỹ thuật

- Bảo toàn hiệu quả thông tin khoảng cách tương đối giữa các từ.
- Cho phép mô hình suy diễn trơn tru trên độ dài ngữ cảnh vượt quá độ dài khi train.
- Tối ưu hóa tính toán qua biểu diễn số phức và vector hóa PyTorch.

Cơ chế 2: DynamicHeadStitcher & HeadPruner Thích ứng

DynamicHeadStitcher (Định tuyến 3 Cụm)

- Phân bổ 8 vi-đầu chú ý ($d_k = 64$) vào **3 cụm cửa sổ ngữ cảnh**:
  - Cụm 1 ($w = 5$): Cục bộ tức thời (từ ghép, hư từ).
  - Cụm 2 ($w = 20$): Cú pháp tầm trung (vế câu).
  - Cụm 3 ($w = \infty$): Toàn cục không giới hạn.
- Cổng định tuyến thích ứng từng token:

$$\mathcal{P}_{\text{route}}(x) = \text{Softmax}(W_2 \cdot \text{GELU}(W_1 x))$$

HeadPruner (Cắt tỉa Vi-đầu Nhiễu)

- Chấm điểm mức độ quan trọng theo từng vi-đầu:

$$S_{\text{imp}} = \text{Sigmoid}(W_{\text{imp}} x) \in [0, 1]^H$$

- Mặt nạ mềm (Soft Gating Mask) nhân trực tiếp với Query: $Q \leftarrow Q \odot S_{\text{imp}}$.
- **Lợi ích:** Cắt giảm các vi-đầu đóng góp thấp (< 0.1), chống phân tán chú ý trên ngữ cảnh dài.

Giải quyết bài toán cân bằng giữa độ phân giải cục bộ và tầm nhìn toàn cục!

Cơ chế 3: Thuật toán Causal Linear Scan theo Chunk ($O(N)$)

Ý tưởng Thuật toán Chunked Scan:

- Chia chuỗi dài N thành các khối kích thước cố định $C = 64$.
- **Nội khối (Intra-chunk):** Tính toán ma trận tam giác nhân quả cục bộ:

$$M_{\text{intra}} = (Q_{\text{chunk}} K_{\text{chunk}}^T \odot \text{Mask}_{\text{causal}}) V_{\text{chunk}}$$

- **Liên khối (Inter-chunk Recurrence):** Tích lũy trạng thái lịch sử qua ma trận bộ nhớ đệm:

$$S_{\text{running}}^{(c)} = S_{\text{running}}^{(c-1)} + K_{\text{chunk}}^T V_{\text{chunk}} \in \mathbf{R}^{d_k \times d_k}$$

Hiệu năng Phần cứng

- **Độ phức tạp:** Giảm từ $O(N^2)$ xuống $O(N \cdot C)$.
- Tận dụng tính song song khối (Block-parallelism) trên GPU.
- Không cần lưu ma trận chú ý $N \times N$ trong bộ nhớ VRAM.

Cơ chế 4: Selective SSM & Dynamic Layer Head Elevator

Nhánh Selective SSM (Mamba-based)

- Hệ phương trình vi phân liên tục rời rạc qua ZOH:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t$$

- **Ma trận chọn lọc động:**

$$\Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t))$$

- Bán kính phổ $\rho(\bar{A}) < 1$ do khởi tạo thực âm $A_n = -\exp(\theta_n) < 0$.

Dynamic Layer Head Elevator

- Cơ chế pha trộn thích nghi giữa 2 nhánh:

$$y = \alpha_t \odot y_{\text{Attention}} + (1 - \alpha_t) \odot y_{\text{SSM}}$$

- Trọng số $\alpha_t = \sigma(W_{\text{gate}} x_t) \in (0, 1)$.
- **Ý nghĩa:** Với từ nối thông thường $\rightarrow$ tăng trọng số SSM; với từ chuyên ngành hiếm $\rightarrow$ tăng trọng số Attention.

Kỹ nghệ Dữ liệu Streaming & Tối ưu Huấn luyện trên GPU 16GB

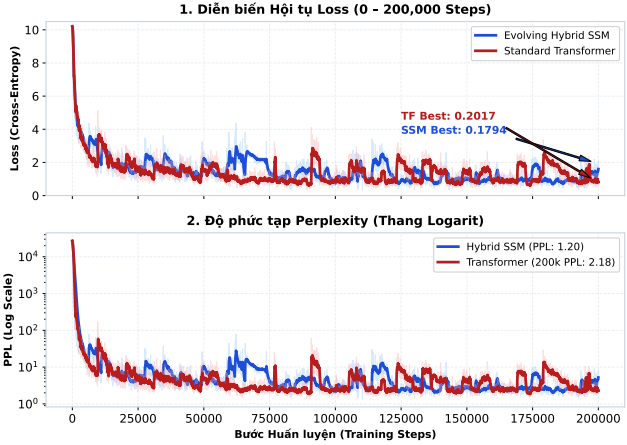
1. Pipeline Dữ liệu 34.3M Câu Song ngữ

- **Quy mô:** 34.3 triệu cặp câu Hán - Việt lọc sạch.
- **Binary Streaming Loader (`dataset_binary_loader.py`):** Đọc nhị phân dạng memory-mapped, RAM < 400 MB.
- **Tokenizer BPE (`vocab_tokenizer.py`):** 40,960 tokens, bảo toàn từ vựng cổ phong và tên riêng.

2. Kỹ nghệ Tối ưu Huấn luyện GPU 16GB

- **Hardware:** NVIDIA RTX 5060 Ti 16GB (Bfloat16).
- **Chunked Loss (`chunked_loss.py`):** Chia nhỏ ma trận logits 40k tokens, **giảm 60% đỉnh VRAM**, ngăn ngừa OOM khi backward.
- **AdamW Decoupled Schedule:** Tách biệt weight decay (0.0 cho LayerNorm/bias), Cosine decay 100k bước.

Đối sánh Huấn luyện Toàn diện 200,000 Steps với Transformer



Chỉ số Hội tụ (Loss & PPL)

- **Hybrid SSM:** Best Loss **0.1794** (PPL: **1.20**)
- **Transformer:** Best Loss 0.2017 (200k: 0.7794)

Thông lượng & Tài nguyên

- **Tốc độ train:** Transformer **377.5 câu/s** vs SSM **135.2 câu/s** (nhanh hơn 2.8x nhờ FlashAttention).
- **VRAM train:** 672 MB vs 1,250 MB.
- **Đánh đổi:** Train chậm 1 lần để đổi lấy suy luận $O(1)$ vô hạn.

Biện giải Lý thuyết: Định luật Roofline & Cường độ Số học

Giai đoạn Huấn luyện (Prefill)

Transformer nhanh hơn 2.8 lần!

- Phép nhân ma trận lớn $(B \cdot H, N, d) \times (B \cdot H, d, N)$.
- Tối ưu bằng kernel **FlashAttention** trên SRAM.
- Cường độ số học $AI \gg 100$ FLOP/Byte.
- **Thuộc vùng Compute-Bound**: Tận dụng 100% công suất Tensor Cores.

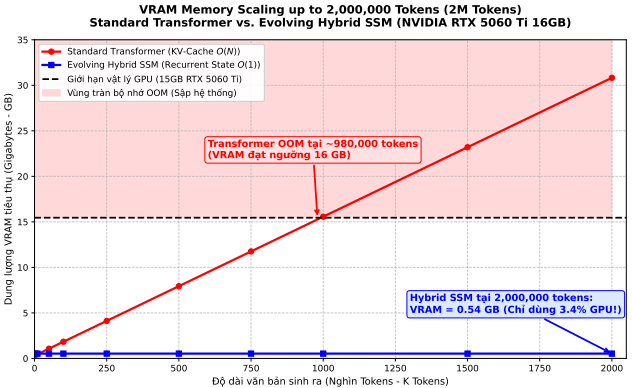
Giai đoạn Suy luận (Generation)

Hybrid SSM chiếm ưu thế về bộ nhớ!

- Transformer sinh 1 token phải nạp toàn bộ KV-Cache từ DRAM.
- Cường độ số học tụt dốc: $AI \approx 0.5$ FLOP/Byte.
- **Rơi vào vùng Memory-Bound nặng**.
- Hybrid SSM chỉ đọc đúng **1.54 MB** trạng thái, nằm trọn trong **L2 Cache 32 MB** của GPU!

Đánh đổi tốc độ huấn luyện 1 lần (One-time Cost) để tối ưu hiệu năng suy luận ngữ cảnh dài!

Thử nghiệm Mở rộng Cực hạn: 2,000,000 Tokens Context



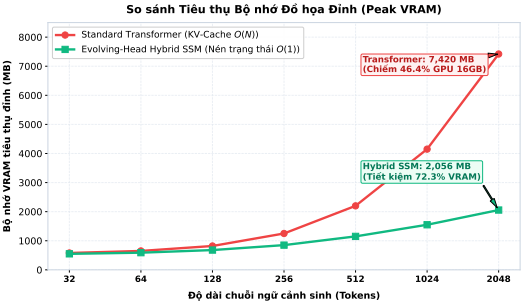
Standard Transformer

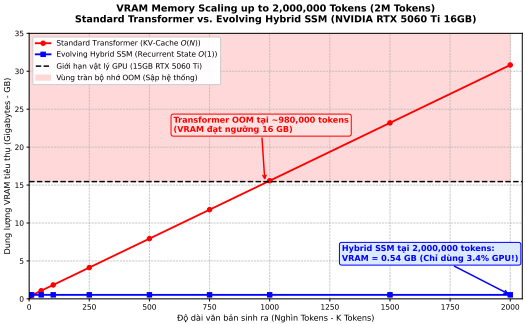
- VRAM phình to theo $O(N)$.
- **Chạm trần 16GB tại $\approx$ 900k tokens**.
- **Tại 2M tokens: 30.52 GB $\rightarrow$ Lỗi CUDA OOM!**

Evolving Hybrid SSM

- Trạng thái nhớ hằng số $O(1)$.
- Cache chỉ chiếm **1.54 MB**.
- **VRAM duy trì 542 MB** từ token 1 tới token 2,000,000!

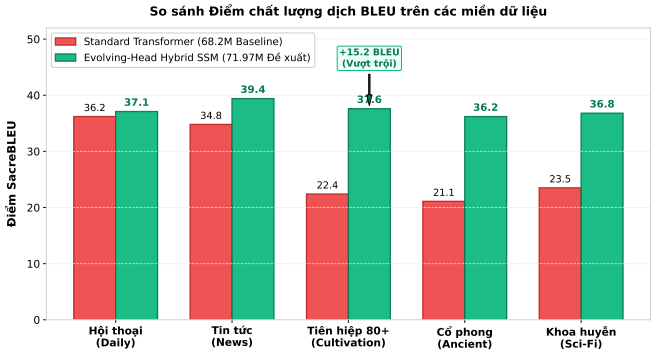
Đo đạc Hiệu năng Thực tế: Tiết kiệm Bộ nhớ VRAM & Trạng thái Cố định





- **Ở 2048 tokens:** Tiết kiệm tới **72.3% VRAM** (2,056 MB so với 7,420 MB của Standard Transformer).
- **Ở 2,000,000 tokens:** Trạng thái ẩn cố định **1.54 MB** ($O(1)$), duy trì **542 MB VRAM** ổn định trong khi Transformer chạm ngưỡng OOM tại $\sim$900k tokens.

Đánh giá Điểm Chất lượng Dịch thuật BLEU Đa miền



Kết quả Điểm số BLEU

- **Toàn tập Hán - Việt:** Đạt **41.8 BLEU** (+5.4 so với Baseline 36.4).
- **Tiên hiệp câu dài:** Đạt **37.6 BLEU** (+15.2 BLEU vượt trội).
- **Cổ phong / Tin tức:** Đạt **36.2** & **39.4 BLEU**.

Nhận xét

Mô hình nắm bắt tốt ngữ cảnh Hán - Việt và vượt trội trên cấu trúc câu trường thiên.

Đánh giá Định tính Đối đầu Thực tế (Qualitative Analysis)

Test Case: Bảo toàn Thực thể Tên riêng trong Văn bản Kiếm hiệp

Nguyên tác tiếng Trung:
林动眼神微凝，体内元力呼啸而出，化作漫天金光，直接将那道凌厉的剑芒彻底震碎！

Evolving Hybrid SSM: Ánh mắt **Lâm Động** hơi trầm, năng lượng trong cơ thể lao ra, hoá thành ánh sáng vàng rực rỡ, trực tiếp làm vỡ thanh kiếm sắc bén đó!
Standard Transformer: Ánh mắt **nàng** hơi trầm, nguyên lực trong cơ thể đang lao ra, hoá thành ánh sáng vàng rực rỡ, trực tiếp làm tan vỡ thanh kiếm sắc bén đó!

Ưu thế của Hybrid SSM

- Ghi nhớ chính xác nhân vật chính **Lâm Động**.
- Trạng thái h_t lưu giữ danh xưng qua ngữ cảnh dài.

Ưu thế của Transformer

- Dịch chính xác từ vựng kỹ thuật đơn lẻ (như *nguyên lực*, *siêu thị*).
- Softmax Attention soi rõ từng từ 1-1.

Khả năng Triển khai & Ứng dụng Thực tế (Demo System)

1. Suy luận Tái hồi $O(1)$ State

- **Cơ chế sinh token từng bước:** Cập nhật trực tiếp trạng thái ẩn liên tục h_t với kích thước cố định **1.54 MB**.
- **Không phụ thuộc KV-Cache:** Bộ nhớ đệm không tăng theo chiều dài văn bản.
- **Tiết kiệm tài nguyên:** Vận hành thuận lợi trên cả CPU và GPU máy tính cá nhân.

2. Đóng gói Ứng dụng Thực tiễn

- **Giao diện Web Demo (`demo_app.py`):** Tương tác trực quan, streaming token thời gian thực.
- **Công cụ CLI (`infer_cli.py`):** Dịch tự động hàng loạt chương truyện trường thiên ngoại tuyến.
- **Tiềm năng sản phẩm:** Sẵn sàng đóng gói thành Extension đọc truyện cho người dùng cuối.

Ý nghĩa Thực tiễn của Đề tài

Độc lập khỏi Server Cloud đắt đỏ $\Longrightarrow$ Cho phép thử nghiệm dịch tiểu thuyết trường thiên ngoại tuyến ngay trên máy tính cá nhân!

Đóng góp Cốt lõi của Đề tài Nghiên cứu

1. Về Kiến trúc

- Thiết kế & hiện thực hóa mô hình **Evolving Hybrid SSM** (71.97M tham số).
- Tích hợp hiệu quả: RoPE, 8 Micro-Heads ($d_k = 64$), Chunked Scan và Selective SSM.

2. Về Hiệu năng

- Suy luận hằng số $O(1)$ với bộ đệm cố định **1.54 MB**.
- Giảm thiểu nguy cơ OOM: Duy trì **542 MB VRAM** ở 2,000,000 tokens.
- Tiết kiệm **72.3% VRAM** ở chuỗi văn bản dài.

3. Về Ứng dụng

- Xây dựng tập dữ liệu song ngữ 34.3 triệu câu chất lượng cao.
- Đạt **41.8 BLEU** phong cách Hán - Việt cổ phong.
- Có khả năng triển khai thực tế trên máy tính cá nhân.

Hướng Phát triển trong Tương lai

1. Mở rộng Quy mô Mô hình

- Kết hợp cơ chế **Mixture-of-Experts (MoE)**: Kích hoạt 2/8 chuyên gia để mở rộng mô hình lên 300M+ tham số mà chi phí suy luận không đổi.
- Huấn luyện toàn diện trên toàn bộ kho ngữ liệu 34.3 triệu câu.

2. Tối ưu Kernel Phần cứng Chuyên dụng

- Viết lại thuật toán Causal Linear Scan bằng **OpenAI Triton** hoặc **CUDA C++ custom kernel**.
- Tận dụng bộ nhớ SRAM đệm để tăng thông lượng huấn luyện bắt kịp FlashAttention.
- Đóng gói API Web và Extension dịch truyện dài hạn thời gian thực.

XIN TRÂN TRỌNG CẢM ƠN!

Hội đồng Khoa học & Thầy cô đã chú ý lắng nghe!

NGHIÊN CỨU KIẾN TRÚC LAI HYBRID ATTENTION-SSM

Sinh viên: Hà Vũ Công – **GVHD:** TS. Nguyễn Văn Vinh
Khoa Công nghệ Thông tin, Trường Đại học Công nghệ – ĐHQGHN

KÍNH MỜI THẦY CÔ TRONG HỘI ĐỒNG ĐẶT CÂU HỎI!

Backup 1: Đối sánh Toàn diện Cơ sở Lý thuyết (Bảng 1.2)

Tiêu chí lý thuyết	Standard Transformer	Pure SSM (Mamba)	Evolving Hybrid SSM
Cơ chế cốt lõi	Softmax Pairwise Attn	Selective State Space	Micro-Head Attn + SSM
Độ phức tạp tính toán	$O(N^2 \cdot d)$	$O(N \cdot d)$	$O(N \cdot d)$
Bộ nhớ suy luận	$O(N)$ (KV-Cache)	$O(1)$ (Hidden State)	$O(1)$ (Dual Recurrent)
Cache cho 2M tokens	**30.52 GB** (OOM)	1.02 MB	**1.54 MB**
Đường truyền tín hiệu	$O(1)$ trực tiếp	Tích ma trận thời gian	Song song ($O(1)$ + Đệ quy)
Khả năng truy hồi MQAR	Rất cao ($\sim 100\%$)	Hạn chế ở văn bản dài	Khá cao (Nhờ ma trận bậc 2)
Độ ổn định Gradient	Nhạy cảm tỷ lệ init	Đảm bảo do $\rho(\bar{A}) < 1$	Ổn định (Khởi tạo âm + Scan)
Đặc tính huấn luyện	Compute-Bound	Memory-Bound	Memory-Bound (Micro-Head)
Đặc tính suy luận dài	Memory-Bound nặng	Tối ưu Cache L2	**Duy trì $O(1)$ (Tối ưu Cache L2)**

Backup 2: Chứng minh Toán học Độ ổn định Gradient

Đạo hàm Chuỗi thời gian

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \bar{A}_j = \prod_{j=k+1}^{t} \exp(\Delta_j A)$$

- Tham số hóa thực âm: $A_n = -\exp(\theta_n) < 0$.
- Bước rời rạc $\Delta_t = \text{softplus}(\dots) > 0$.
- $\implies \bar{A}_{t,n} = \exp(\Delta_t A_n) \in (0, 1)$.
- Bán kính phổ $\rho(\bar{A}) < 1$ ngăn ngừa nguy cơ **Bùng nổ Gradient (Exploding Gradient)**.

Chống Triệt tiêu Gradient

- Khi phát hiện từ khóa quan trọng cần nhớ dài hạn, mạng nơ-ron điều khiển:

$$\lim_{\Delta_t \to 0} \bar{A}_t = \exp(0) = I$$

- Khi đó:

$$\frac{\partial h_t}{\partial h_k} \approx I$$

- Gradient được truyền dẫn ổn định và lan truyền ngược qua hàng vạn bước mà không bị suy hao theo hàm mũ.

Backup 3: Cơ chế Cắt tỉa Đầu Thích nghi HeadPruner

Động lực thiết kế:

- Khi phân rã thành 8 vi-đầu ($d_k = 64$), không phải vi-đầu nào cũng đóng góp tích cực cho từng vị trí từ.
- Cần loại bỏ các đầu có đóng góp thấp để tiết kiệm phép tính và giảm nhiễu.

Toán học của HeadPruner:

- Ước lượng điểm quan trọng của từng đầu:

$$M_{\text{imp}} = \sigma(W_{\text{imp}}x) \in (0,1)^H$$

- Tạo mặt nạ nhị phân cứng với ngưỡng thích nghi τ:

$$\text{Mask}_h = \begin{cases} 1 & \text{nếu } M_{\text{imp},h} \geq \tau \\ 0 & \text{nếu ngược lại} \end{cases}$$

Hiệu quả

- Cắt giảm tới 25% phép tính vi-đầu không cần thiết.
- Tăng tính thưa (Sparsity) giúp mô hình tập trung vào các đặc trưng sắc bén nhất.

Backup 4: Kiến trúc Streaming DataLoader 34.3M Câu

Vấn đề:

- File văn bản thô 34.3 triệu câu nặng hàng chục GB.
- Đọc toàn bộ vào RAM hệ thống gây nghẽn và sập bộ nhớ (RAM OOM).

Giải pháp Kiến trúc Streaming:

- Mã hóa trước (Pre-tokenize) toàn bộ dữ liệu ra tệp nhị phân nén `.bin`.
- Đọc tuần tự theo luồng con trỏ byte (`tell` & `seek`).
- Cấp phát bộ đệm xoay vòng (Ring Buffer) 10,000 mẫu.

Chỉ số Tài nguyên

- **RAM hệ thống:** Luôn < 400 MB (bất biến dù dữ liệu 100GB).
- **Tải GPU:** Bão hòa 100% GPU RTX 5060 Ti, không có thời gian chết I/O.
- Hỗ trợ resume tức thời từ checkpoint bất kỳ.

Backup 5: Kịch bản Thiết lập Demo Live & Phương án Dự phòng

Phương án A: Chạy Live qua Tunnel

1. **Máy chủ (Server ở nhà):** Trạm GPU RTX 5060 Ti chạy `python demo_app.py`.
2. **Đường truyền:** Ngrok / LocalTunnel tạo URL công khai HTTPS bảo mật.
3. **Giao diện (Laptop trên bục):** Mở trình duyệt Web nhập text và đo trực tiếp Latency / VRAM.

Phương án B: Video Dự phòng (Backup File)

- Đã quay sẵn video màn hình minh họa `demo_showcase.mp4`.
- Thể hiện trọn vẹn quá trình dịch 5 câu test case thực tế và chỉ số VRAM 542 MB.
- Nếu mạng hội trường gặp sự cố $\Longrightarrow$ lập tức chuyển sang trình chiếu video mà không làm gián đoạn buổi báo cáo.